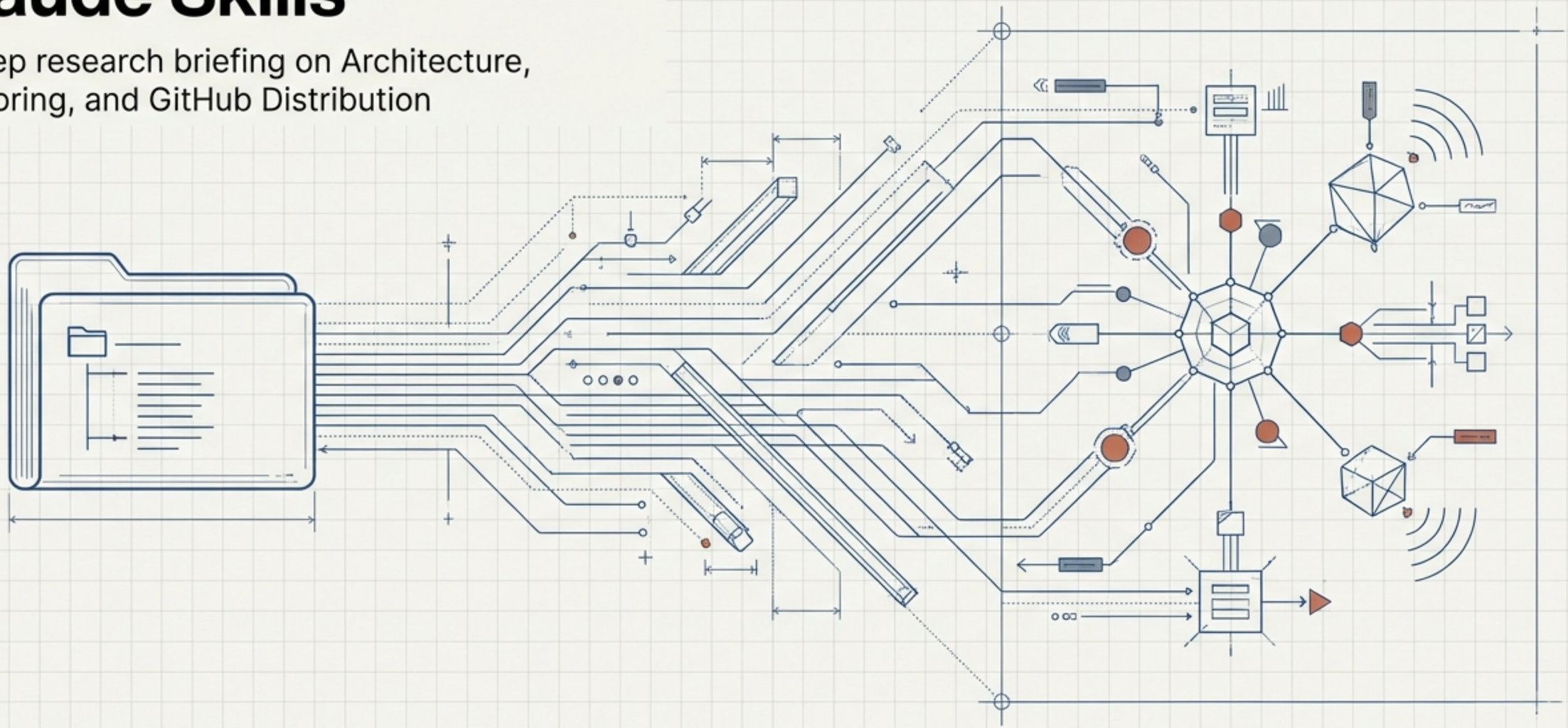
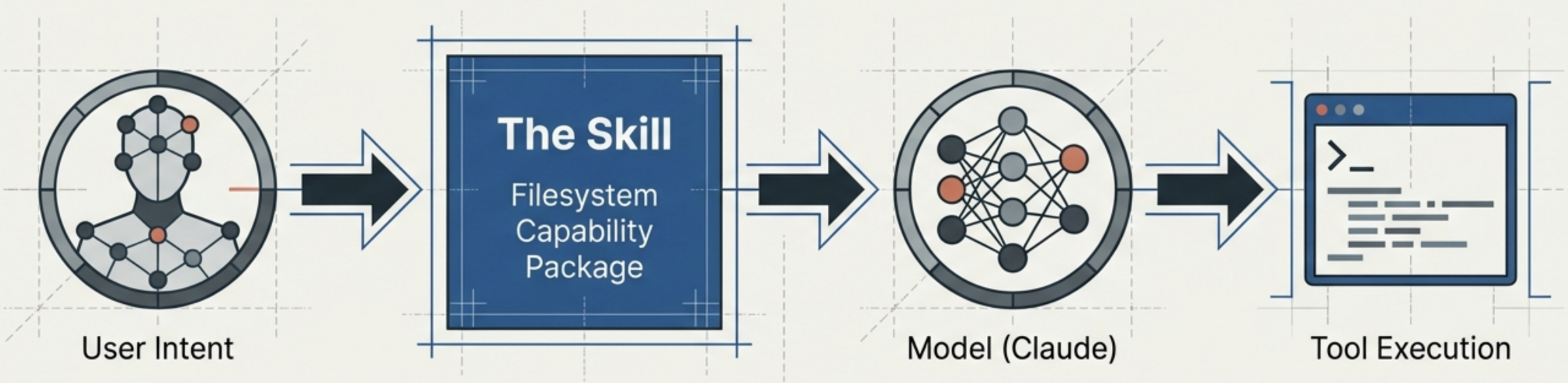
Developing Production-Grade Claude Skills

A deep research briefing on Architecture, Authoring, and GitHub Distribution



Based on the agentskills.io open standard and Anthropic engineering guidance.
Classification: Technical Briefing / Architecture

The Paradigm Shift: From Chatting to Onboarding



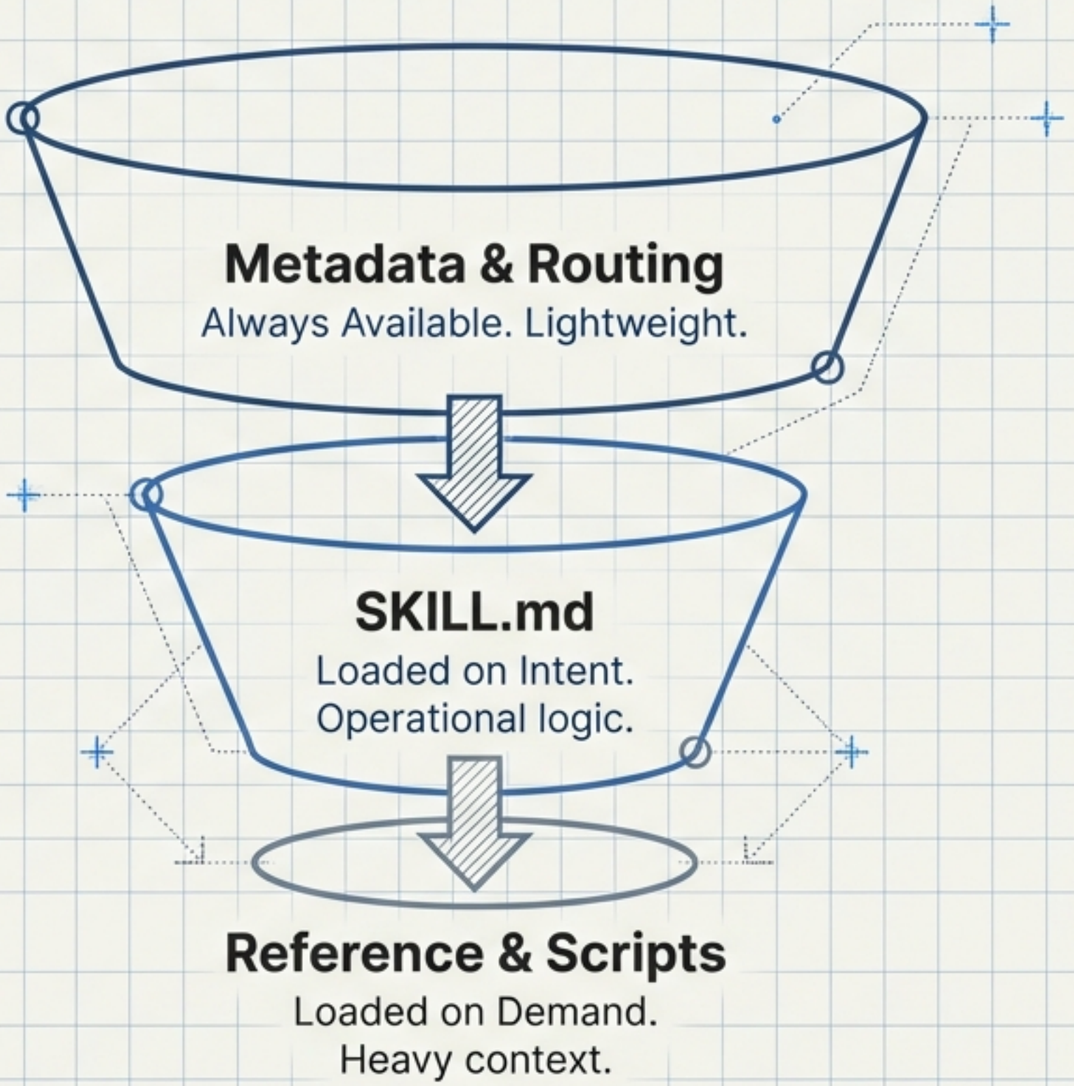
The Mental Model

A Skill is not a prompt; it is a modular, filesystem-based capability package. Think of it as installing software rather than conversing with a chatbot.

Skills as Onboarding Guides

The Skill provides the map; Claude drives the car. The goal is to organise content so the model can navigate to details as required, rather than flooding the context window immediately.

Optimising for Context: Progressive Disclosure



The 500-Line Rule

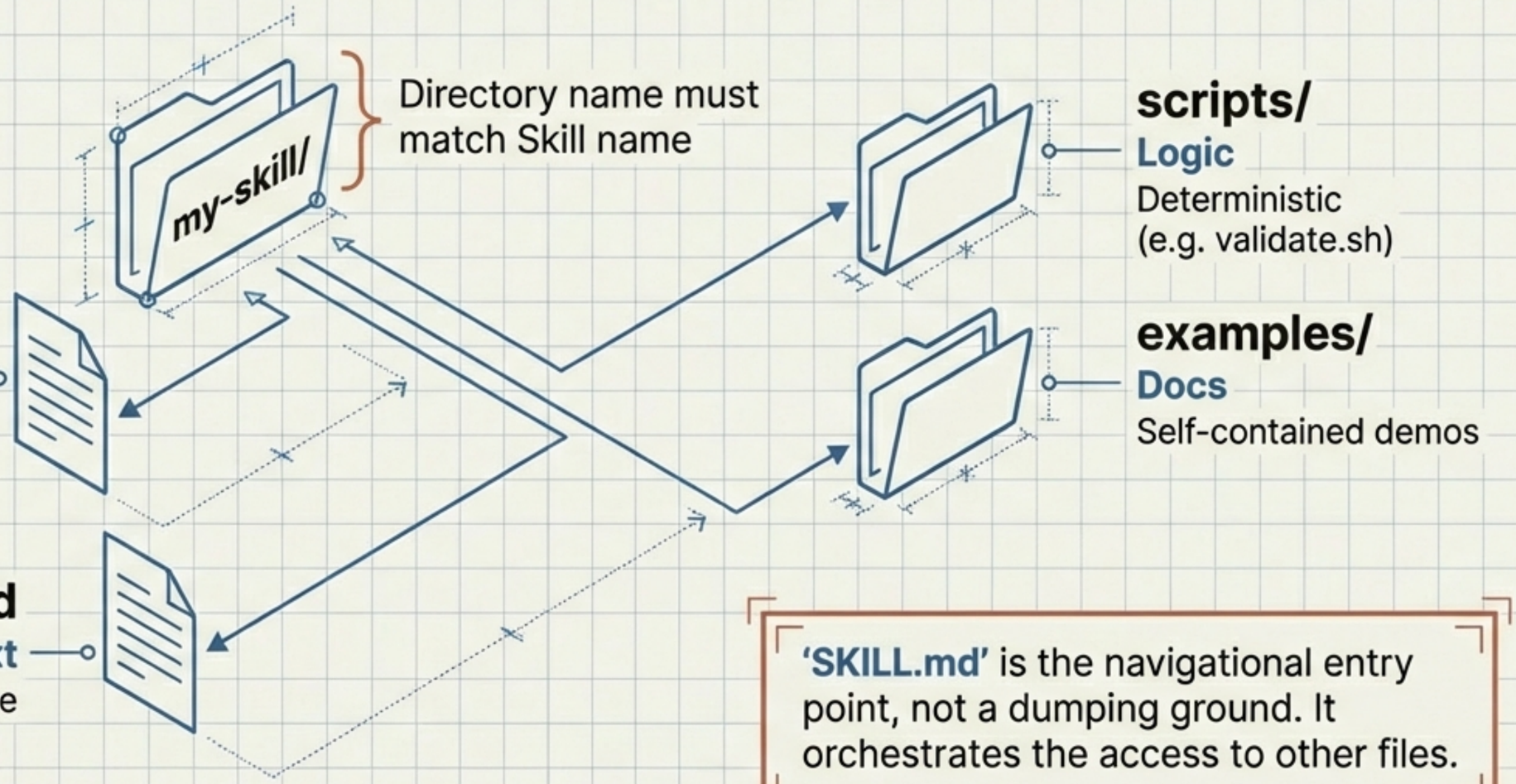
Keep ‘SKILL.md’ navigational and under ~500 lines. Move dense documentation and templates to ‘reference.md’ to avoid context penalties.

One Format, Three Distinct Runtimes

Portability does not mean Sync. Users must manage lifecycles separately per surface.

	Claude Code (CLI)	Claude API	Claude.ai
Installation	Local/Project folder (`~/.claude/skills`)	Upload to `/v1/skills` (Zip/Tuple)	Zip upload via Settings
Network	Local machine access	None (Isolated Container)	Variable / Browser-based
Features	Shell pre-processing, sub-agents	No runtime package installation	Per-user manual management

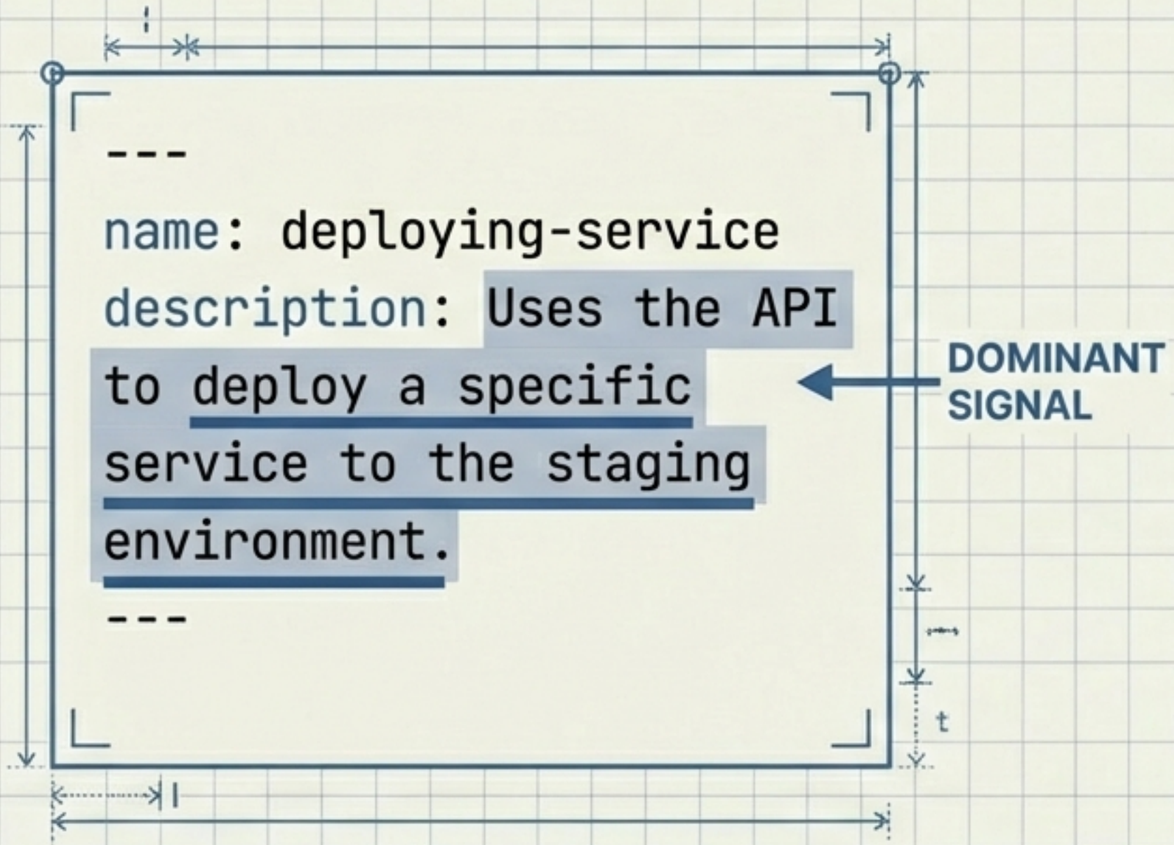
Anatomy of a Skill: The Filesystem is the API



'SKILL.md' is the navigational entry point, not a dumping ground. It orchestrates the access to other files.

The Routing Layer: Metadata is King

The '**description**' field is the dominant signal for loading.



DO / DON'T: Writing Descriptions

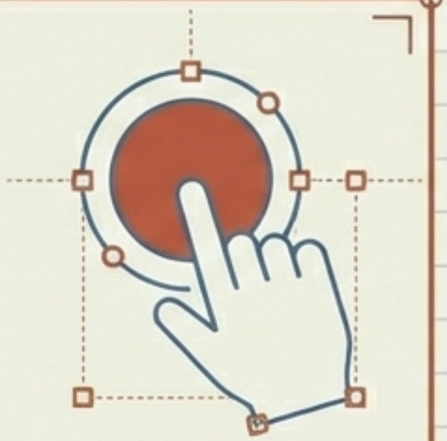
	DO	DON'T
Perspective	✓ Write in 3rd person ("Uses the API...")	✗ System-prompt style ("You are a helper...")
Content	✓ State strictly what it does and when to use it.	✗ Be vague or overlap with other skills.
Naming	✓ Use Gerund forms ("deploying-service").	✗ Use generic verbs ("deploy").

Invocation Gates and Control Flow

Preventing the ‘Auto-Pilot’ problem for task-based skills.

Manual Only

```
disable-model-
invocation: true
```



You can invoke it, but Claude cannot trigger it autonomously.

Critical: This hides the description from context to prevent hallucinated triggers. Use for high-stakes ops (e.g., deploy).

Background Only

```
user-invocable:
false
```

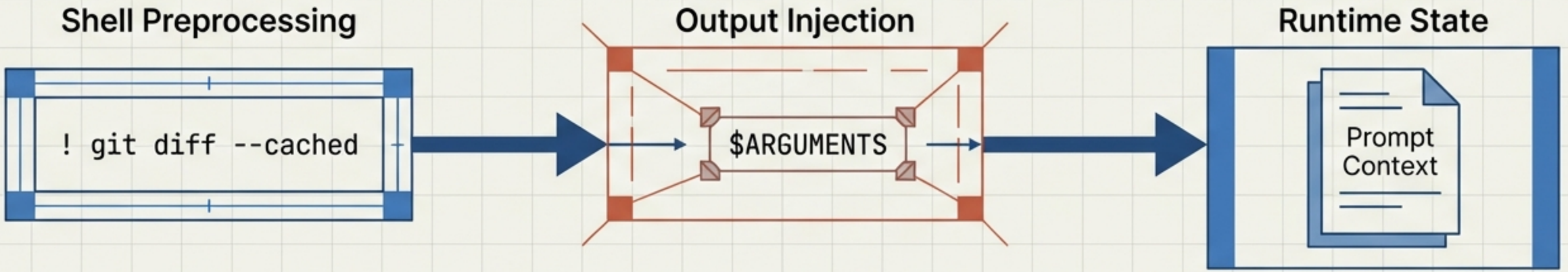
The model can call this to assist a query, but it is hidden from the user’s slash-command menu.

Use for sub-routines.

Workflow Adaptors: Dynamic Context Injection

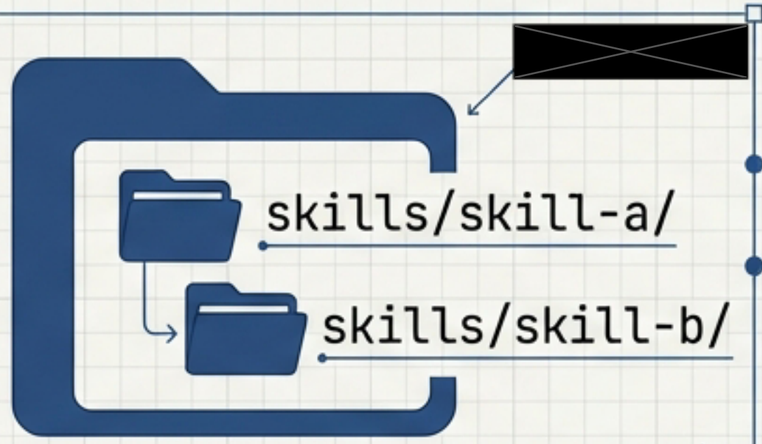
Bridging the gap between static text and repo state. Models don't know the current state (diffs, logs) until you tell them.

```
input:
  - name: changes
    description: The current git diff
    type: string
    content: ! git diff --cached
```



Repository Strategy: Monorepo vs. Polyrepo

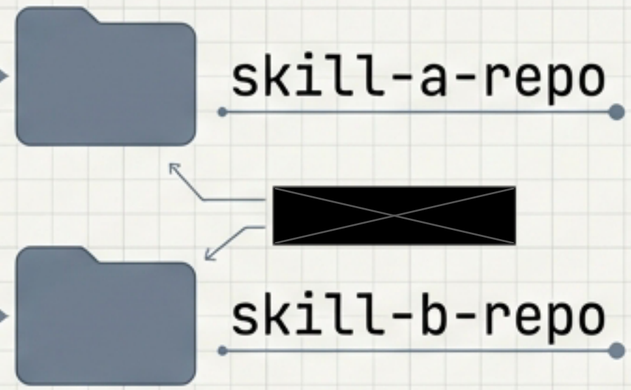
The Toolkit (Monorepo)



- **Structure:** `skills/skill-a/`, `skills/skill-b/`
- **Pros:** Shared CI, cohesive standards.
- **Cons:** Harder to vendor single skills.

Skill Library

The Product (Polyrepo)

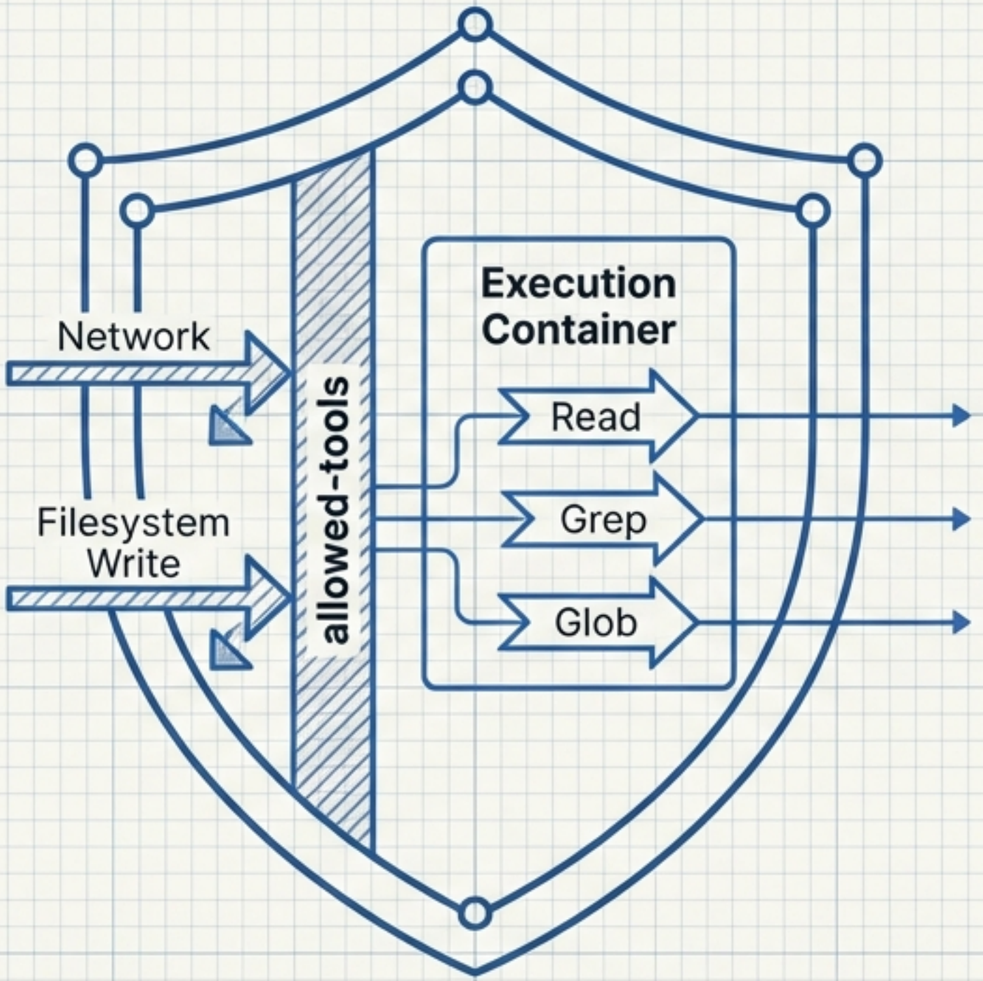


- **Structure:** One repo per skill.
- **Pros:** Clean install, SemVer, distinct maintainers.
- **Cons:** High maintenance overhead.

Constraint: The directory name must match the Skill name field for cross-tool interoperability.

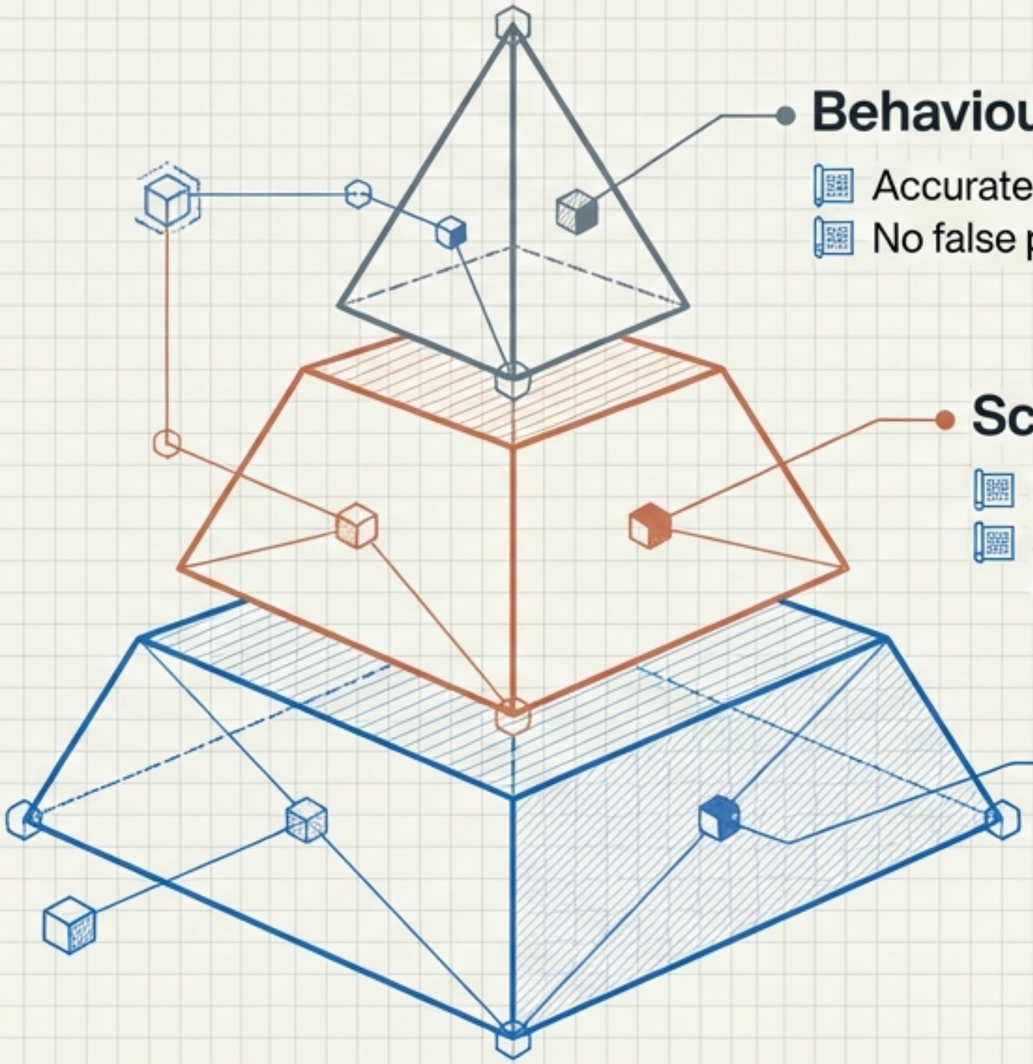
Security: Tooling Boundaries & Least Privilege

- **Capability-Based Model:** Constrain what a skill can touch.
- **Example:** A “Reader” skill should not have access to “Edit” or “Run”.



Supply Chain Risks: Be wary of typosquatting and malicious scripts masquerading as standard tools.

The Testing Matrix: Beyond “It Worked Once”



Behavioural Triggering

- Accurate invocation?
- No false positives?

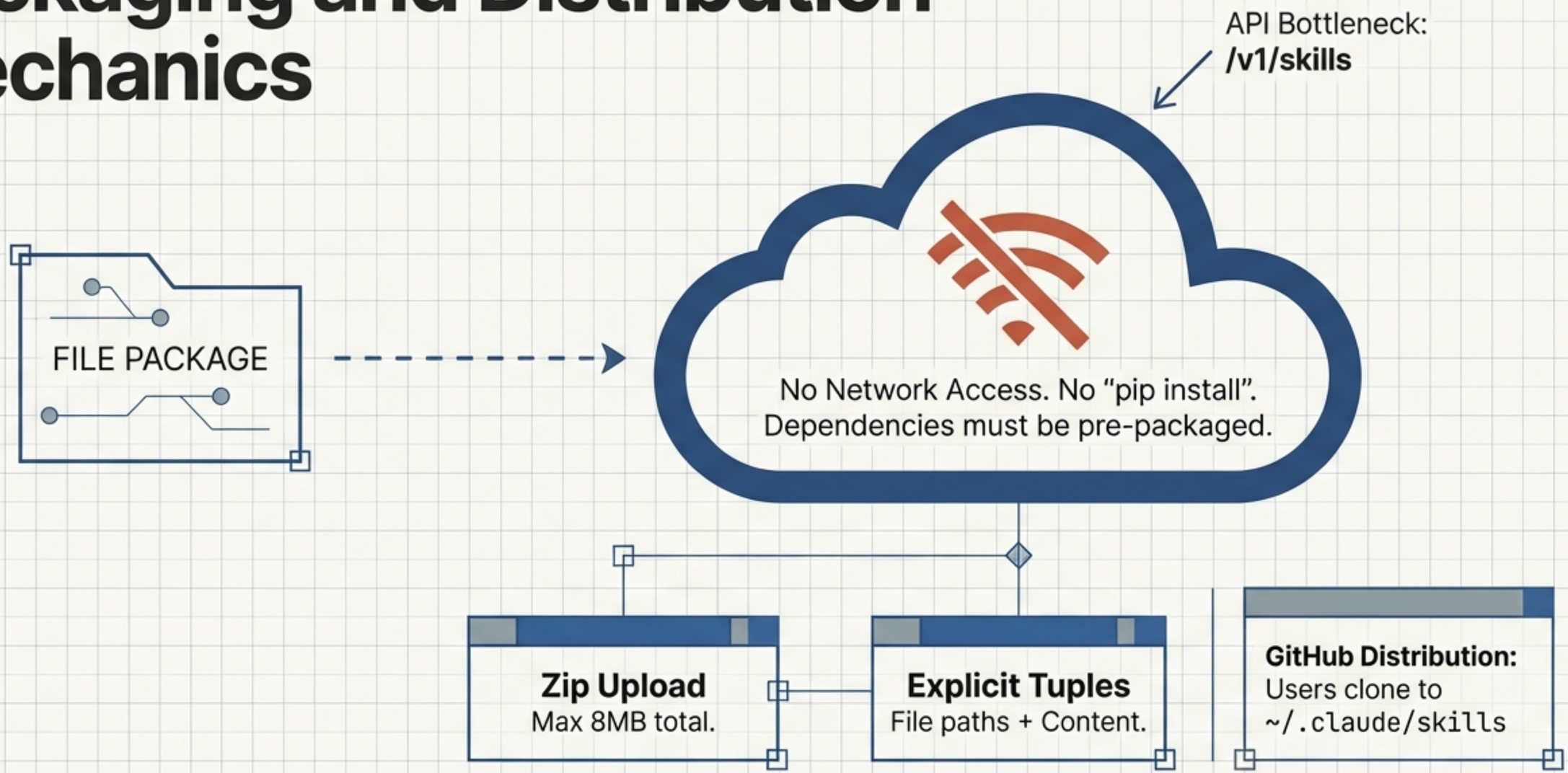
Script Determinism

- Environment independent?
- Scripts must not assume local packages exist (Silent Failures).

Spec Conformance

- Valid YAML? Folder name matches name field?
- (Reference: agentskills.io spec)

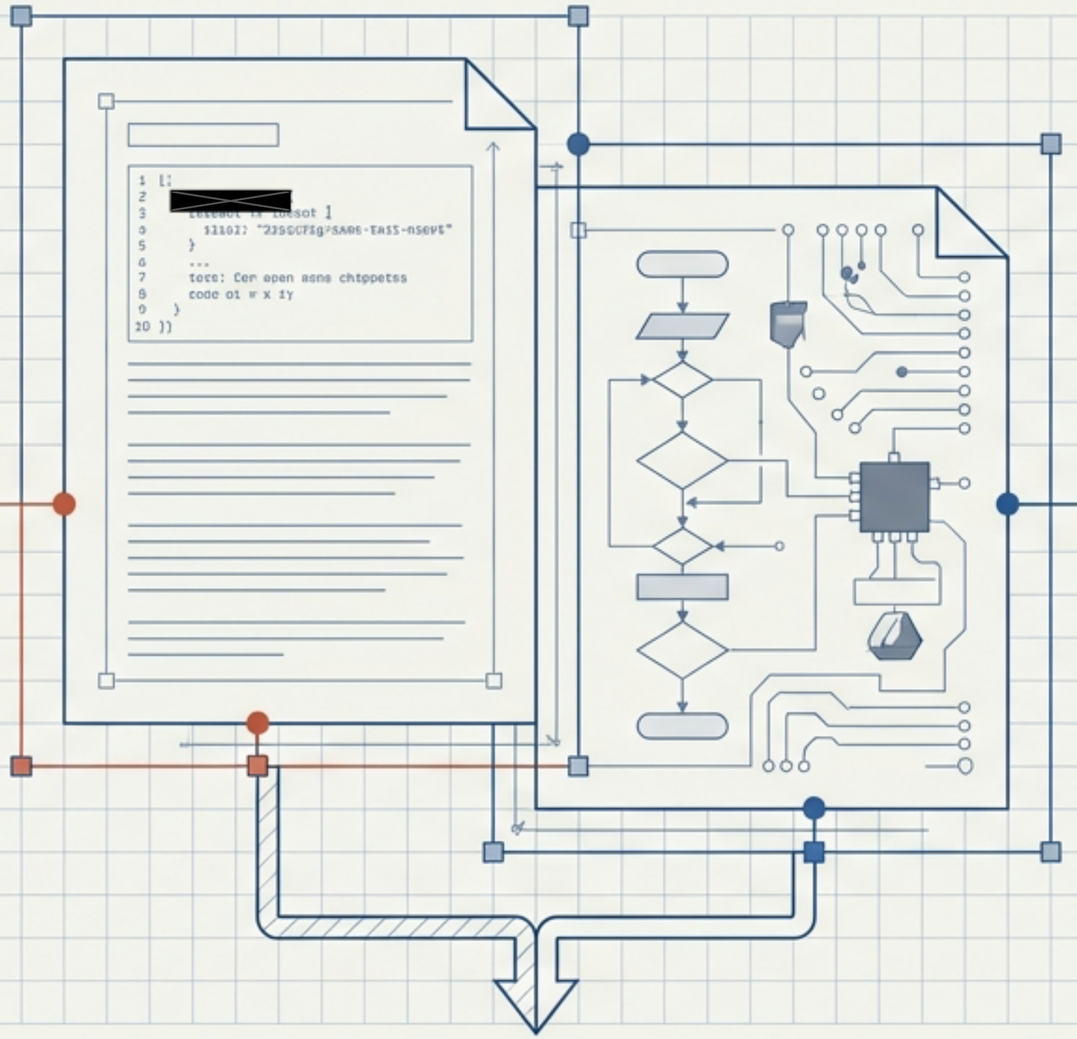
Packaging and Distribution Mechanics



Licensing for the Open Web: Text vs. Software

Markdown / Prompts
Inter

CC-BY (Attribution).
Creative works.



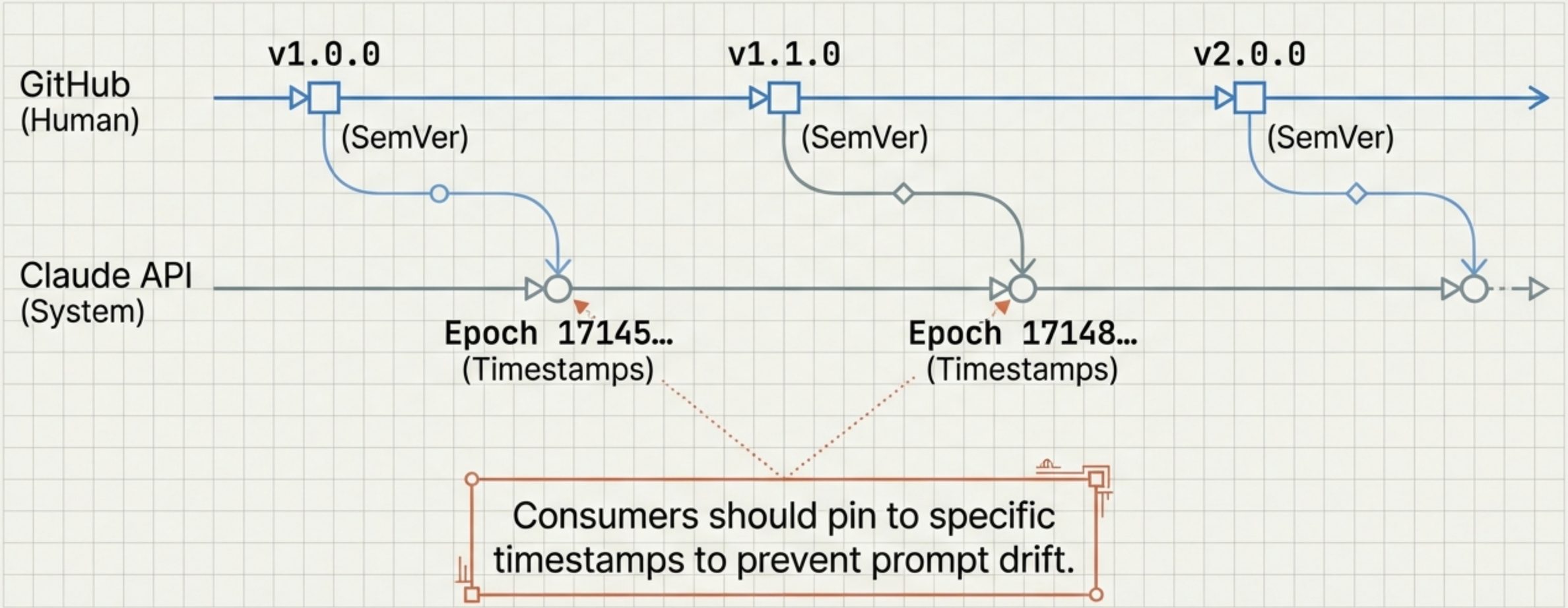
Scripts / Code
Inter

MIT / Apache 2.0.
Executable software.

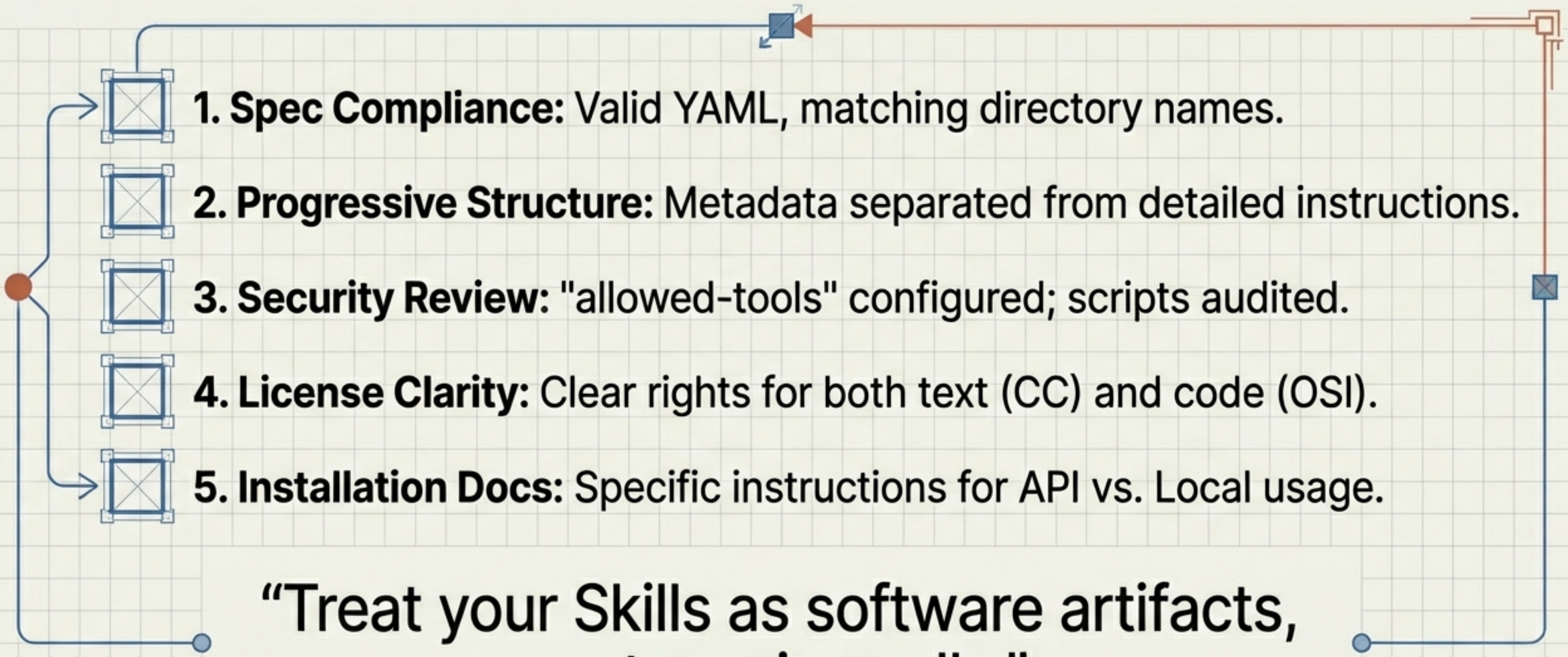
Dual-Licensing Strategy ensures enterprise compliance.
Text is content; Scripts are code.

Release Engineering and Versioning

Managing drift: Pin in Production, Float in Dev.



Summary Checklist: The Definition of ‘Done’



1. **Spec Compliance:** Valid YAML, matching directory names.
2. **Progressive Structure:** Metadata separated from detailed instructions.
3. **Security Review:** "allowed-tools" configured; scripts audited.
4. **License Clarity:** Clear rights for both text (CC) and code (OSI).
5. **Installation Docs:** Specific instructions for API vs. Local usage.

“Treat your Skills as software artifacts,
not magic spells.”